Experiment No. 1

* Aim: Feature Transformation
A) To use PCA Algorithm for dimensionality reduction. You have a dataset that includes measurement for different variables on Wine (alcohol, ash, magnesium and so on). Apply PCA algorithm & transform this data so that most variation in the measurement of the variables are captured by a small no. of principal Component. Dataset link: https://media.geeksforgeeks.org/wp-content/wine.csv.

* objective:
the main objective of this experiment are to:

1. Understand and implement the PCA algorithm for dimensionality reduction
2. Transform the wine dataset to lower-dimensional space using PCA.
3. Visualize the transformed data to observe patterns & differences between red & white wines.

* Theory:
Principal Component Analysis (PCA): principal Component analysis is a dimensionality reduction technique used to reduce large set of variables to a smaller one.

Goals of PCA:
- Reduce dimensionality
- Retain as much as variance as possible.
- find new axes that are linear combination of original features.

When to Use PCA:

- You have high dimensional data.
- Many features are correlated.
- You want to visualize data in 2D/3D

Example Dataset (2D):

Let's say we have the following 4 data point with two features:

X (Height in cm)	Y (weight in cm)
160	55
165	65
170	60
175	70

We want to reduce this to 1D using PCA.

step 1: Standardize the data.

PCA is sensitive to scale, so we standardize each feature.

Let's compute meand and standard deviation.

Mean height = (160 + 165 + 170 + 175) / 4 = 167.5

Mean weight = (55 + 65 + 60 + 70) / 4 = 62.5

Standardize X (height)	Standardize Y (weight)
(160 - 167.5) = -7.5	(55 - 62.5) = -7.5
-2.5	2.5
2.5	-2.5
7.5	7.5

then divide by standard deviation (assume std = 6.45)

Z_x	Z_y
-1.16	-1.16
-0.39	0.39
0.39	-0.39
1.16	1.16

step 2: Compute covariance matrix.

$$Conv(X,Y) = \frac{1}{n-1} \sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})$$

for standardize data,

$$\text{Cov matrix} = \begin{bmatrix} 1 & 0.93 \\ 0.93 & 1 \end{bmatrix}$$

Step 3: Compute eigen values and eigen vectors.

- Eigen vectors: Direction of new axes
- Eigen values: How much variance each axis Capture.

For this matrix, Suppose we get:

- PC1 (first eigenvector) = [0.71, 0.71], eigen value = 1.93
- PC2 (second eigenvector) = [-0.71, 0.71], eigen value = 0.07

Interpretation

- PC1 Captures most variance (~96%)
- PC2 Captures little variance (~4%)

Step 4: Project data into principal Component

Now we project original standardize data onto PC1 axis:

Use dot product:

for point [-1.16, -1.16] & PC1 = [0.71, 0.71]

$z = [-1.16, -1.16] \cdot [0.71, 0.71] = -1.64$ $z = [-1.16, -1.16]$

dot $[0.71, 0.71] = -1.64$ $z = [-1.16, -1.16] \cdot [0.71, 0.71] = -1.64$

Do this for all points, and now you have a 1D representation that captures most important message in your data.

Applications:

PCA has a application in various domains:

- Face recognition (Eigenfaces)
- Data Compression (e.g. image storage)
- Noise reduction (e.g., EEG data)

* Input:

The input for this experiment is the wine dataset, which includes measurement for different variables related to wine characteristics.

* Output:

The output of this experiment includes:
1. Transformed dataset with reduced dimension using PCA
2. Visualization of the transformed data to observe the distinction between red and white wines.

* Conclusion:

through the application of PCA, we successfully demonstrate how to transform high dimensional wine data into a lower dimensional space while retaining important variation.

* Outcome:

The outcome of this experiment is transformed datasets with reduced dimensions using PCA, as well as visualization that help observe the separation between red and white wines in transformed space.

Experiment No.2

* Aim: Regression Analysis.

A) predict the price of the uber ride from given pickup point to the agreed point drop-off location. perform following task:
1. pre process the data
2. Identify outliers
3. check the Correlation
4. Implement linear regression and ridge, Lasso regression model.
5. Evaluate the models and compare their respective sources like R_2, RMSE etc.

* objective:

The main objective of this experiment are to:

1. Preprocess the dataset to make it suitable for regression analysis.
2. Detect and handle outliers that might affect the predictive models.
3. Asses the Correlation between features to understand relationship.
4. Apply linear regression, ridge regression, and Lasso regression model for price prediction:
5. Evaluate and Compare the performance of the models using metrices such as R2 & RMSE.

* Theory:

Regression analysis is a statistical method used to model the relationship between a dependent variable (target) and one or more independent variables.
It allows us to:
- Predict values
- Understand the strength and nature of relationship.

Type	When to use
Linear regression	Linear relationship between variable.
Multiple linear regression	More than one independant variables.
polynomial regression	Relationship is nonlinear but can be modded by powers.
Ridge/Lasso regression	To prevent overfitting in complex models.

Linear regression basics

- equation of Line : $y = mx + b$

where, y = predicted value
x = input feature
m = slope
b = intercept.

- General linear Regression formula:

$$y = B_0 + B_1x_1 + B_2x_2 + \dots + B_nx_n + \varepsilon$$

B_0 is intercept
B_{ij} are the coefficient
ε is the error term.

Predict the weight based on their height using linear regression.

Dataset:

Height (cm)	Weight (kg)
150	52
160	58
170	65
180	70
190	78

Step 1: plot the data
Before anything, visualize it.
You'd likely see a trend: as height increases, weight also increases.

Step 2: Fit a linear regression model.

We want to fit a line of best fit

Weight = m x Height + b

Let's say after calculating, we find:

Weight = 0.6 x Height - 40

Step 3: Use the model to predict

if someone is 175 cm tall:

Weight = 0.6 x ~~175~~ 175 - 40 - 105 - 40 = 65 kg

So the predicted weight is 65 kg

Suppose:

- $R^2 = 0.97$ → model explains 97% of the variations
- RMSE = 2.5 kg → predictions deviate on average by 2.5 kg

That's a good model.

* Applications:

	Use Case	Independent variables	Dependent variable
1.	House price prediction	Area, rooms, locations	Price
2.	Weather forecast	Humidity, wind speed, season	Temperature
3.	Uber fare estimate	Distance, duration, time of day	Fare.

* Input:

The input for this experiment is the Uber ride price dataset, which contains features related to pickup and drop-off locations, time and ride attributes.

* output :-

The output of this experiment includes:

1. Trained regression models (linear, ridge & Lasso) for predicting Uber ride prices.
2. Evaluation metrics (R_2, RMSE) for each model, facilitating comparison.

* Conclusion:

Through the application of regression analysis techniques, we successfully demonstrate prediction of uber ride prices based on provided features. the experiment highlighted the effectiveness of different regression models and their performance evaluation.

* outcome:

The outcome of this experiment is a set of trained regression models (linear, ridge & Lasso) capable of predicting Uber ride prices. Additionally we obtain insights into the relative performance of this models using evaluation metrics.

Experiment No. 3

* Aim: Implement K-Nearest Neighbours algorithm on social network ad dataset. Compute Confusion matrix, accuracy, error rate, precision and recall on given dataset.

* Problem statement:

Implementation of K-Nearest neigbours algorithm ad dataset for classification and Compute Confusion matrix, accuracy, recall, error rate and precision on the given datasets.

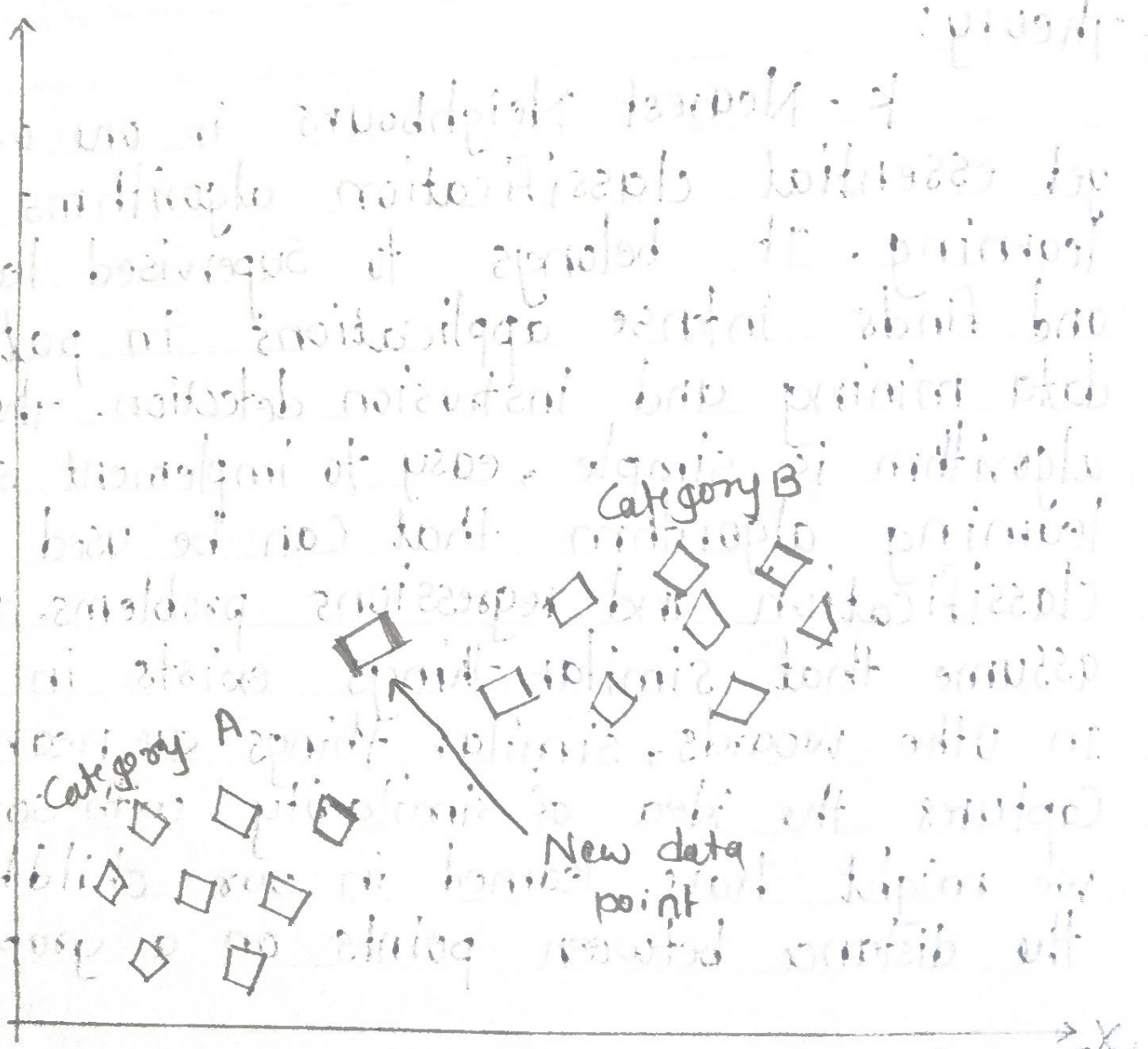
* objective: Apply the KNN classification algorithm to classify the data with appropriate labels.

* Theory:

K-Nearest Neighbours is one of the most basic yet essential classification algorithms in machine learning. It belongs to supervised learning domain and finds intense applications in pattern recognition, data mining and instrusion detection. the K-Nearest Neigbours algorithm is simple, easy to implement supervised machine learning algorithm that can be used to solve both classification and regressions problems. the KNN algorithm assume that similar things exists in close proximity. In othe woards, similar things are near each other. KNN Captures the idea of similarity with some mathematics we might have learned in our childhood - calculating the distance between points on a graph.

Example:

Suppose, we hav an image of creatures that looks


X_2
Category B
Category A
New data point
X_1

similar to Cat and dog. but we want to know either it is a Cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measures.

Importing libraries and datasets

python libraries make it very easy for us to handle the data and perform typical and complex tasks with a single line of code.

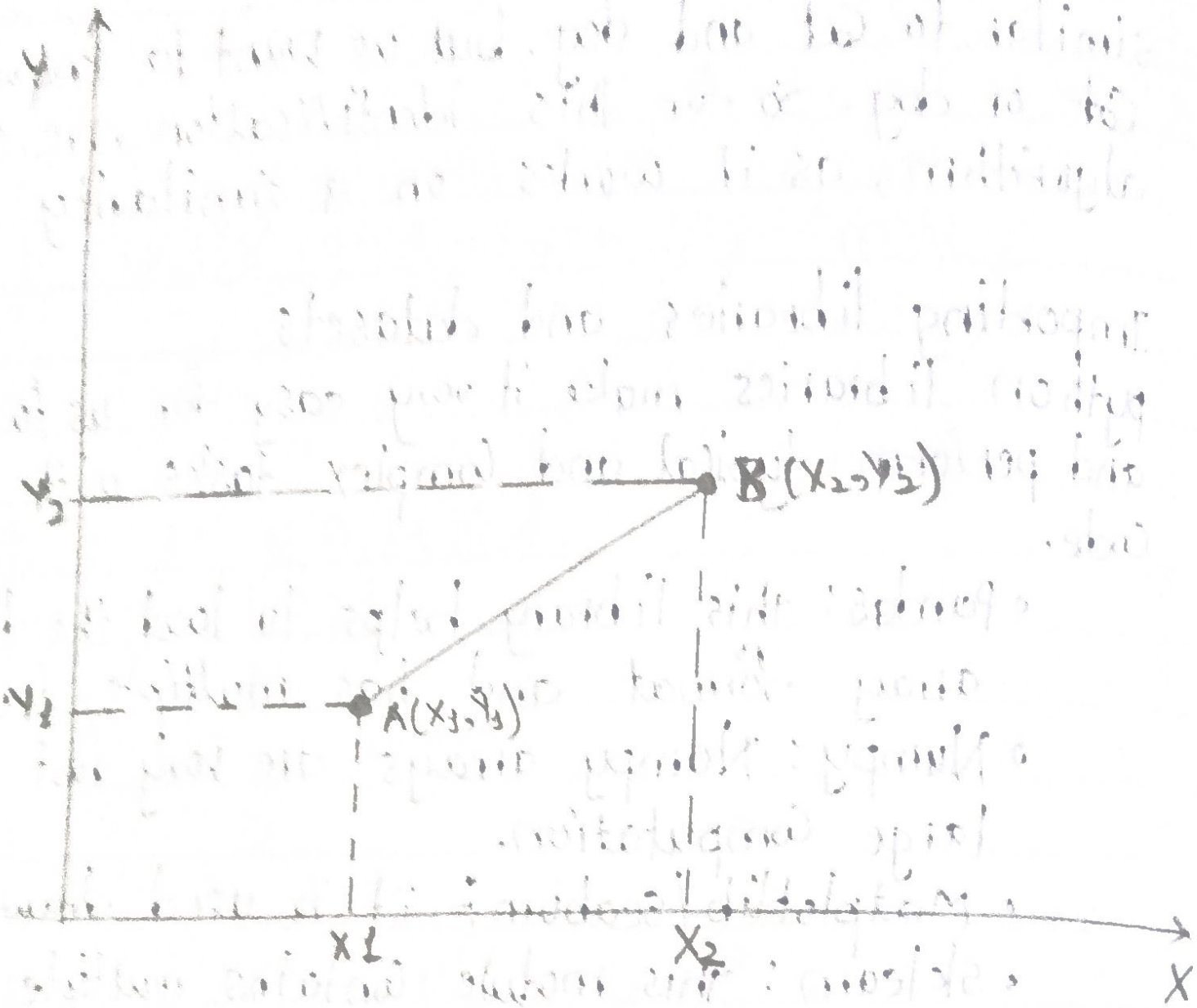
- Pandas: this library helps to load the data frame in a 2D array format and has multiple functions.
- Numpy: Numpy arrays are very fast and can perform large computation.
- Matplotlib/seaborn:- it is used draw visualizations.
- sklearn: this module contains multiple lines having pre implemented functions.

KNN algorithm:

Step 1: Select the no. k of the neighbours
step 2: Calculate the Euclidean function & distance of k number of neigbours.
step 3: Take the k-Nearest Neighbours as per calculate Euclidean distance.
step 4: Among these k-neighbours, count no. of data points.
Step 5: Assign the new data points to the category for which the no. of the neighbours is maximum.
Step 6: our model is ready.

Suppose we have a new data point and we need to put it in the required Category. Consider the following image.

- Firstly we will choose the number of neighbours, so will choose


Y
Y2
Y1
B(X2,Y2)
A(X1,Y1)
X1
X2
X

the k=5

- Next we will calculate the Euclidean distance between data points. the Euclidean distance is the distance between two points. which we have already suited in geometry.

Euclidean distance between A_1 & $B_1 = \sqrt{(X_2-X_1)^2 + (Y_2-Y_1)^2}$

By calculating the Euclidean distance we got the nearest neighbours as three nearest Neighbours in category A & two nearest neighbours in category B.

* Advantage of KNN:
- it is easy to understand & implement.
- It can also handle multiclass classifications problems.
- useful when data does not have a clear distribution.
- It works on non-parametric approach.

* Disadvantage of KNN:
- Sensitive to the noisy feature in the dataset.
- Computationally sensitive and expensive.
- It can biased in the imbalanced dataset.
- Sometimes normalization may required.

* Conclusion:

Thus, we have successfully implemented KNN algorithm on given data set for classification and computed Confusion matrix, accuracy, error rate, precision and recall on given dataset.

Experiment No. 4

* Aim: Clustering Analysis

A) Implement k-means clustering on Iris.csv dataset. Determine the number of clusters using the below method.

* Objective:

The main objective of this experiment is to understand and implement k-means clustering and the elbow method for selecting the appropriate number of clusters in the given Iris dataset.

* Theory:

k-means clustering is an unsupervised learning algorithm used to group similar data points into k distinct, non overlapping clusters, based on feature similarity.

It tries to minimize the distance between point within a cluster and maximize the distance between different clusters.

How k-mean works step-by-step.

1. choose k (no. of clusters)
2. Initialize k centroids randomly
3. Assign each point to nearest Centroid.
4. Recalculate Centroids as the average of points in each cluster.
5. Repeat step 3-4 until:
   - Centroids stops moving or
   - A max no. of iteration is reached.

Example: Grouping 6 points into 2 clusters.

Sample data (x, y coordinates):

Point	X	Y
A	1	2
B	1	4
C	1	0
D	10	2
E	10	4
F	10	0

We can clearly see:
- points A, B, C are near (1, *)
- Points D, E, F are near (10, *)

We expect two clusters.
- cluster 1: A, B, C
- cluster 2: D, E, F

points	Coordinates.
A	(1, 2)
B	(1, 4)
C	(1, 0)
D	(10, 2)
E	(10, 4)
F	(10, 0)

step:
Choose k=2 cluster and initalize Centroids
we randomly picks two initial centroids.
Centroid 1 (c1) : Point A → (1, 2)
Centroid 2 (c2) : Point D → (10, 2)

step 1: Assign each point to nearest Centroid by Euclidean distance.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Point	Dist to C1=(1,2)	Dist to C2=(10,2)	Assign cluster
A (1,2)	0	$\sqrt{(10-1)^2+(2-2)^2} = 9$	C1 (cluster 1)
B (1,4)	$\sqrt{(1-1)^2+(4-2)^2} = 2$	$\sqrt{(10-1)^2+(4-2)^2} = 9.22$	C1
C (1,0)	$\sqrt{(1-1)^2+(0-2)^2} = 2$	$\sqrt{(10-1)^2+(0-2)^2} = 9.22$	C1
D (10,2)	9	0	C2
E (10,4)	$\sqrt{(10-1)^2+(4-2)^2} = 9.22$	$\sqrt{(10-10)^2+(4-2)^2} = 2$	C2
F (10,0)	$\sqrt{(10-1)^2+(0-2)^2} = 9.22$	$\sqrt{(10+10)^2+(0-2)^2} = 2$	C2

Step 2: Calculate new centroids

- cluster 1 points A(1,2), B(1,4), C(1,0)

$$C1_{new} = \left(\frac{1+1+1}{3}, \frac{2+4+0}{3}\right) = (1, 2)$$

- cluster 2 points D(10,2) E(10,4) F(10,0)

$$C2_{new} = \left(\frac{10+10+10}{3}, \frac{2+4+0}{3}\right) = (10, 2)$$

Step 3:- Check for convergence

Since centroids didn't change, the algorithm converges.

Iteration	Centroid 1	Centroid 2	Cluster 1 points	Cluster 2 Points
Init	(1,2)	(10,2)	–	–
1	(1,2)	(10,2)	A(1,2), B(1,4),	D(10,2), E(10,4),
			C(1,0)	F(10,0)

* Applications:

1. Customer segmentation: Identifying distinct customer groups based on behaviour.
2. Image segmentation: Grouping similar regions in images for analysis.
3. Anomly detection: Identifying usual pattern in Data.
4. Document clustering: Grouping similar document for info. retrival.

* Input:

The input for this experiment is the Iris.csv dataset which contains measurement of iris flower's features (sepal length, sepal width, petal length & width)

* output:

The output of this experiments includes:

1. clusters: the identified clusters of data points after applying the k-means algorithm.
2. optimal no. of clusters: the no. of clusters determined using the elbow method.

* Conclusion:

Through the implementation of k-means clustering on Iris dataset, we successfully demonstrate how this technique can group similar iris flower based on their measurement. The elbow method allowed us to determine the optimal number of clusters.

* Outcome:

The outcome of this experiment is set of cluster formed through k means clustering, along with the optimal number of cluster identified using the elbow method.

Experiment No. 5

* Aim: Ensemble Learning

Implement Random Forest classifier model to predict the safety of car. Link: https://www.kaggle.com/dataset/car-evaluation-data set

* objective:

the main objective of this experiment is to utilize ensemble learning techniques, specially the Random Forest classifior, to create a predictive model that can assess the safety of cars.

* theory:

Random Forest classifier:

Random Forest classifier is an ensemble learning method used for classification. It builds multiple decision trees during training and outputs the mode of the classes predicted by individual trees.

Why Random Forest?

- Helps reduce overfitting Compared to a single decision tree.
- Handles High dimensional data well.
- Robust to noise and outliers.
- Works well with large dataset.

How it coorks?

1. Bootstrap Sampling: Randomly samples data points with replacement to create different subsets.
2. Decision Trees: Train the decision tree on a each subset.
3. Random feature selection: At each split of tree, only random subset of feature is Considered.

* Applications:

1. Finance: Credit risk assement and Fraud detection.

2. Medicine: Disease prediction and patient diagnosis.

3. Natural Language processing: sentiment analysis.

4. Image processing: obiect detection and recognition.

* Input:

The input for this project includes the data set provided in link: https://www.kaggle.com/dataset/elikplim/dataset. the data set Contain features related to car specification and attributes.

* output:

The output of this experiment is predictive model build on Random forest classifier. For each Car in dataset the model will provide the prediction of its safety level.

* Conclusion:

Through the implementation of the Random Forest classifier on the car safety dataset, we successfully demonstrate how ensemble learning can be used to predict the safety levels of cars.

* outcome: the outcome of this experiment is a trained Random forest classifier model that can predict the safety levels of cars within a certain degree of accuracy.

Experiment NO:6

* Aim:

Builds a Tic-Tac-Toe game using Reinforcement learning in Python by using following tasks:

a. setting up the enviroment
b. Defining the Tic-Tac-Toe game.
c. Building the Reinforcement learning model.
d. Training the model.
e. Testing the model.

* Objective : To learn use of Reinforcement learning.

* Software Requirements: open source Python, programming tool like Jupyter notebook, Pycharm, Spyder.

* Operating System Recommended: 64-bit open source linux or its derivative.

* Theory: Introduction

In this, we will learn how to creat an agent that learns to play the game by trial and error, taking actions and receiving rewards or penalties depending on wheather the action led to win, loss or draw.

* Prerequisite :-

- Basic knowledge of python.
- Familarity with fundamentals of machine learning, specially Reinforcement learning.
- Familarity with Tensorflow library.

Rules of the Game

- The game is to be played between two people.
- one of the player chooses 'O' and the other 'X' to mark their respective cells.
- The game starts with one of the players and the game ends when one of the player has one whole row/ Column/digonal filled with his respective character.
- If no one wins, then game is said to be draw.

* Implementation: In our program the moves taken by the Computer and the human are chosen randomly. We use rand() function for this.

What more can be done in the program?

the program is in not played optimally by both sides because the moves are chosen randomly. The program can be easily modified so that both players play optimally. Also the program can be modified such that the user himself gives the input.

Winning strategy - A interesting fact.

If both players play optimally then it is destined that you will never lose ("although the match can still be drawn"). It doesn't matter whether you play first or second in another ways - "two expert players will always draw"! Isn't this interesting?

* Conclusion:

Hence we study Reinforcement learning.